

Proximal Policy Optimization on CartPole-v1: Clipped vs. Unclipped Surrogate Objectives

Anuj Rajan Lalla (B22AI061) Abhinav Swami (B22AI003)
Swaksh Patwari (B22AI065)

Abstract

We implement Proximal Policy Optimization (PPO) [1] from scratch in PyTorch and evaluate it on the **CartPole-v1** environment from the Gymnasium library. The implementation follows the standard actor-critic recipe with a shared two-layer MLP backbone, a categorical policy head, a scalar value head, on-policy rollout collection, Generalized Advantage Estimation (GAE) [2], and mini-batch PPO updates. We compare the clipped surrogate objective L^{CLIP} against the unclipped importance-sampled objective L^{IS} under matched hyperparameters over three random seeds. Our main finding is that *clipping trades peak performance for stability* at the 60k-step budget used here: the clipped agent has a very small across-seed standard deviation (10.4) but a moderate final mean return (152.8); the unclipped agent reaches a higher mean return (376.0) but with an order-of-magnitude larger across-seed standard deviation (175.4). Per-update KL diagnostics explain the contrast directly: the clipped objective keeps KL in a narrow band, while the unclipped surrogate allows large KL spikes. Hyperparameter sweeps at a single seed and 40k environment steps show non-monotonic behaviour for most knobs (clip range ϵ , GAE λ , and entropy coefficient c_e) and a clean positive trend for learning rate and PPO epochs within the ranges tested, underlining how noisy single-seed reinforcement learning can be. The critic’s value function learns a sensible landscape that peaks near the upright state and decays as the pole tilts or spins. Rendered rollouts at four training checkpoints illustrate the non-monotonic nature of on-policy learning with a single seed.

1 Introduction

Policy gradient methods optimize a parameterized policy $\pi_\theta(a \mid s)$ by ascending $\mathbb{E}[\nabla_\theta \log \pi_\theta(a \mid s) A(s, a)]$. A practical obstacle is that rollouts are expensive, so we would like to run several gradient steps on the same batch of data. Once θ drifts from the θ_{old} that generated the data, however, an importance-sampled correction is required, and the ratio $\pi_\theta/\pi_{\theta_{\text{old}}}$ can explode, destabilizing training.

Trust Region Policy Optimization (TRPO) [3] addresses this by constraining the KL divergence between the old and new policy at every step, but the constrained optimization is expensive to implement. Proximal Policy Optimization (PPO) [1] achieves a similar effect with a much simpler recipe: clip the importance ratio inside the policy objective so that the policy is never rewarded for moving far from θ_{old} . PPO has become a de facto standard for on-policy reinforcement learning.

In this work we implement PPO from scratch with a clear separation of concerns (policy network, value network, rollout generator, advantage estimator, loss function, update routine), compare the clipped and unclipped objectives, run sweeps over the principal hyperparameters, and visualize both the learned value function and the rendered behaviour of the trained policy.

2 Method

2.1 Environment

CartPole-v1 is the classic control task in which the agent applies a discrete left or right force to a cart to keep a hinged pole balanced upright. The observation is a four-dimensional vector $(x, \dot{x}, \vartheta, \dot{\vartheta})$ (cart position, cart velocity, pole angle, pole angular velocity). The reward is +1 per timestep, and an episode terminates when the pole falls past a threshold angle, the cart leaves the track, or the step cap (500) is reached. The maximum episodic return is therefore 500.

2.2 Actor-Critic Network

We use a small multilayer perceptron with a shared two-layer tanh trunk (64 hidden units) and two heads: a policy head that outputs logits for a categorical distribution over the two actions, and a value head that outputs a scalar $V_\phi(s)$. Orthogonal initialization is used throughout, with a small gain of 0.01 on the policy head to keep the initial policy close to uniform (a standard PPO detail).

$$\pi_\theta(a \mid s) = \text{Categorical}(\text{softmax}(\text{logits}(s))), \quad V_\phi(s) \in \mathbb{R}.$$

The `act()` routine is called during rollout collection (no gradients, samples an action and returns $(a, \log \pi_\theta(a \mid s), V_\phi(s))$). The `evaluate()` routine is called during the update (with gradients, recomputes log-probabilities, values, and policy entropy under the current θ).

2.3 Rollout Collection

The agent runs in the environment for a fixed number of timesteps ($T = 1024$ per rollout), recording $(s_t, a_t, r_t, \log \pi_{\theta_{\text{old}}}(a_t \mid s_t), V_\phi(s_t), \text{done}_t)$ at each step. When an episode ends we reset and continue, so a single rollout usually spans several episodes. For the final (possibly mid-episode) state we also compute $V_\phi(s_T)$, which is used as the bootstrap target for GAE.

2.4 Generalized Advantage Estimation

Advantages are computed by sweeping backwards through the rollout buffer:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1})(1 - d_t) - V_\phi(s_t), \quad (1)$$

$$\hat{A}_t = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}, \quad (2)$$

where $d_t \in \{0, 1\}$ is the episode-end flag (the $(1 - d_t)$ mask correctly zeroes out bootstrapping across episode boundaries). The returns-to-go used as regression targets for the value head are $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$. The parameter λ interpolates between the one-step TD residual ($\lambda = 0$, low variance but biased and heavily dependent on V_ϕ) and the Monte-Carlo return ($\lambda = 1$, unbiased but high variance). We use $\lambda = 0.95$ as the default.

2.5 The Two Objectives

Let $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ be the importance ratio. The *unclipped* importance-sampled surrogate is

$$L^{\text{IS}}(\theta) = \mathbb{E}_t \left[r_t(\theta) \hat{A}_t \right]. \quad (3)$$

This is unbiased at $\theta = \theta_{\text{old}}$, but nothing prevents $r_t(\theta)$ from growing arbitrarily large, so a single sample with a large ratio and positive advantage can push θ into a region where the old data is no longer informative. The *clipped* PPO objective replaces this with

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right]. \quad (4)$$

The min is what makes clipping work: it only takes effect when it reduces the objective, so the policy is never rewarded for moving far from θ_{old} . We use $\varepsilon = 0.2$ as the default. The two objectives are coded in a single function governed by a boolean flag, so every other component of the algorithm (network, rollout loop, GAE, value loss, optimizer step) is strictly identical in the comparison.

2.6 Full Loss and Update

The complete loss combines the policy surrogate, a value-function regression term, and an optional entropy bonus:

$$L(\theta, \phi) = -L^{\text{CLIP}}(\theta) + c_v \|V_\phi(s_t) - \hat{R}_t\|^2 - c_e H[\pi_\theta(\cdot | s_t)], \quad (5)$$

with $c_v = 0.5$ and $c_e = 0.0$ by default. Each rollout is used for $K = 10$ epochs of mini-batch updates (mini-batch size 64), with per-rollout advantage standardization (subtracting mean, dividing by standard deviation) and global gradient-norm clipping at 0.5. Three diagnostics are recorded per update: the approximate KL divergence $\widehat{\text{KL}} = \mathbb{E}[\log \pi_{\theta_{\text{old}}} - \log \pi_\theta]$ (how far each update moved the policy), the clip fraction $\mathbb{E}[\mathbb{1}\{|r_t - 1| > \varepsilon\}]$ (share of samples where the clip was active), and the value-function MSE.

3 Experimental Setup

Table 1: Default training configuration.

Parameter	Value
Environment	CartPole-v1 (Gymnasium)
Observation / action dim.	4 / 2
Hidden units (per layer)	64, tanh activation
Total environment steps	60,000 (main comparison), 40,000 (sweeps)
Steps per rollout T	1024
Discount γ	0.99
GAE λ	0.95
Clip range ε	0.20
PPO epochs K	10
Mini-batch size	64
Value loss coefficient c_v	0.50
Entropy coefficient c_e	0.00
Optimizer	Adam (lr = 3×10^{-4})
Gradient-norm clip	0.5
Seeds (main comparison)	$\{0, 1, 2\}$

A single run of reinforcement learning is noisy, so the main comparison (clipped vs. unclipped) is averaged over three seeds. The same three seeds are used for both variants so the two agents start from matching initial conditions. Hyperparameter sweeps use a single seed (seed 0) and a shorter horizon (40k steps) to keep runtime modest. The 40k, single-seed regime is noisy enough that readers should treat individual numbers as noisy estimates and focus on orders of magnitude rather than small gaps. Training was performed on CPU (wall-clock approximately 200 seconds per 60k-step run).

4 Results: Clipped vs. Unclipped PPO

4.1 Learning Curves

Figure 1 shows the per-seed learning curves for the two variants, with individual seeds as thin lines and the across-seed mean as a thick line with a ± 1 standard-deviation band. The clipped agent produces a tight across-seed band: individual seeds stay close to the mean and the curves are comparatively smooth. The unclipped agent produces a much wider band: some seeds reach the 500-return cap while others collapse back to short episodes, and individual curves oscillate sharply across iterations.

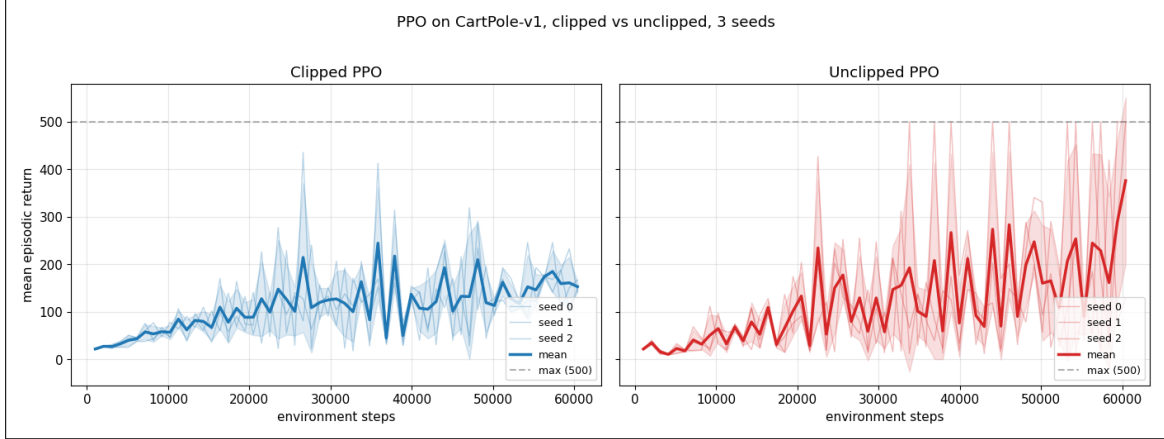


Figure 1: Per-seed learning curves for clipped PPO (left) and unclipped PPO (right) on **CartPole-v1**. Thin lines are individual seeds; the thick line is the across-seed mean with a ± 1 std band; the dashed grey line marks the 500-return cap. The clipped band is noticeably tighter, indicating more consistent behaviour across seeds.

Figure 2 overlays the two mean curves directly. The final summary statistics (Table 2) tell the quantitative story.

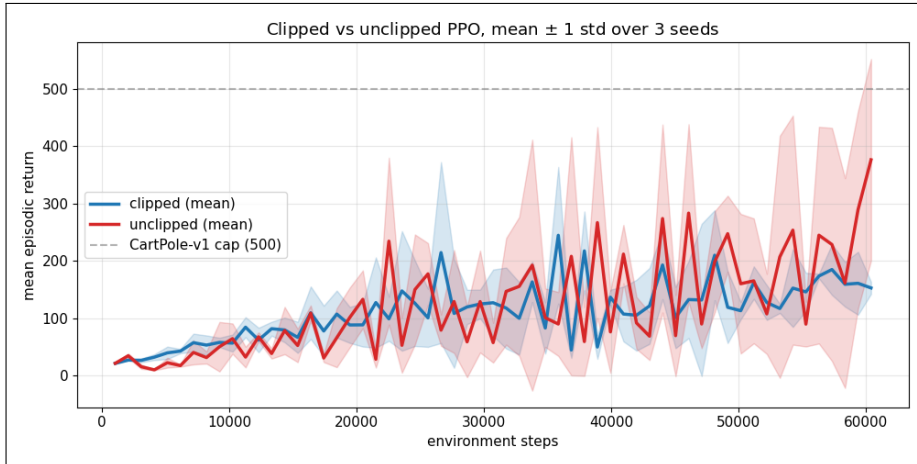


Figure 2: Clipped vs. unclipped PPO, mean ± 1 std across 3 seeds. The unclipped band is wider at every point in training. The unclipped mean trends higher than the clipped mean, but with substantially more across-seed and iteration-to-iteration variability.

Table 2: Final-iteration summary statistics across the 3 seeds (60,000 environment steps). Numbers are taken directly from the printed output of the notebook.

Metric	Clipped	Unclipped
Final mean return (across seeds)	152.8	376.0
Best across-training mean return	244.5	376.0
Across-seed std at final iteration	10.4	175.4
Iteration-wise std (mean over training)	42.4	76.2

The numbers tell a nuanced story. The unclipped variant actually attains a higher final mean return (376.0 vs. 152.8) at this particular training budget, but its across-seed standard deviation (175.4) is more than sixteen times larger than the clipped variant’s (10.4). Looking at the individual seed-0 training log confirms the pattern: the unclipped run oscillates between runs of $R = 500$ (iterations 45 and 55) and collapses to $R \approx 14$ (iteration 30 with a KL spike of 1.21), whereas the clipped run stays in a much tighter band around $R \in [30, 210]$ throughout. The takeaway is therefore not “clipping achieves higher returns” but “clipping trades peak performance for stability”: the clipped variant gives a tight, predictable band at the cost of not capitalizing on lucky updates that happen to work. For a task as short as **CartPole-v1**, a single lucky seed of the unclipped variant can reach 500, which is why its mean curve looks superficially better; the failure modes visible in the per-seed curves are the relevant signal.

4.2 Diagnostics

Figure 3 explains the difference in stability. The KL panel shows the average per-update KL divergence. The clipped agent keeps KL in a narrow band centred around 0.02; the unclipped agent produces much larger and noisier KL values, with occasional spikes (the seed-0 log shows one of order 1.21 at iteration 30). The clip-fraction panel shows the share of samples that the clip was actively bounding: a moderate fraction (roughly 20 to 30%) for the clipped run, and identically zero for the unclipped run by construction. The value-loss panel is similar between the two, which is the expected behaviour: the value head is trained identically in both setups, so any difference in policy performance must come from the policy update itself.

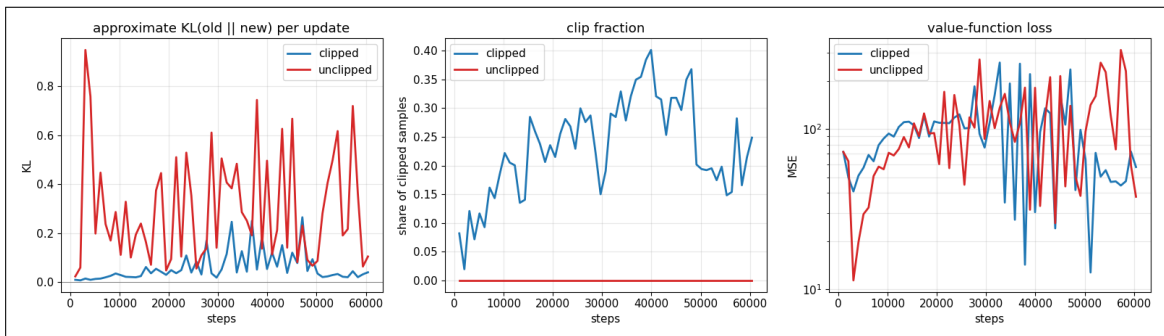


Figure 3: Diagnostics averaged over 3 seeds for both variants. Left: approximate per-update KL divergence. Clipped PPO keeps KL small and steady; the unclipped surrogate shows much larger and noisier KL values. Middle: clip fraction (always zero for the unclipped run). Right: value-function MSE on a log scale, essentially identical for the two variants, which isolates the policy-update step as the source of the performance difference.

5 Hyperparameter Ablations

All sweeps in this section use the clipped objective, a single seed (seed 0), and 40k environment steps. Each sweep varies one hyperparameter while holding the rest at their defaults from Table 1. A single seed and a short horizon together produce substantial noise in the final-return estimate, so individual numbers should be interpreted as noisy samples rather than precise rankings.

5.1 Clip Range ε

Table 3 lists the final mean returns. The ranking is not monotonic: the smallest clip range ($\varepsilon = 0.05$) does well, the default ($\varepsilon = 0.2$) is middling at this horizon, and the loosest clip ($\varepsilon = 0.5$) has the highest final return in this single-seed sweep. This last result is consistent with the main comparison, in which the unclipped variant (which is the limit $\varepsilon \rightarrow \infty$) had a high but highly variable return.

Table 3: Effect of clip range ε (clipped PPO, seed 0, 40k steps).

ε	Final return
0.05	161.3
0.10	58.5
0.20	139.6
0.50	206.5

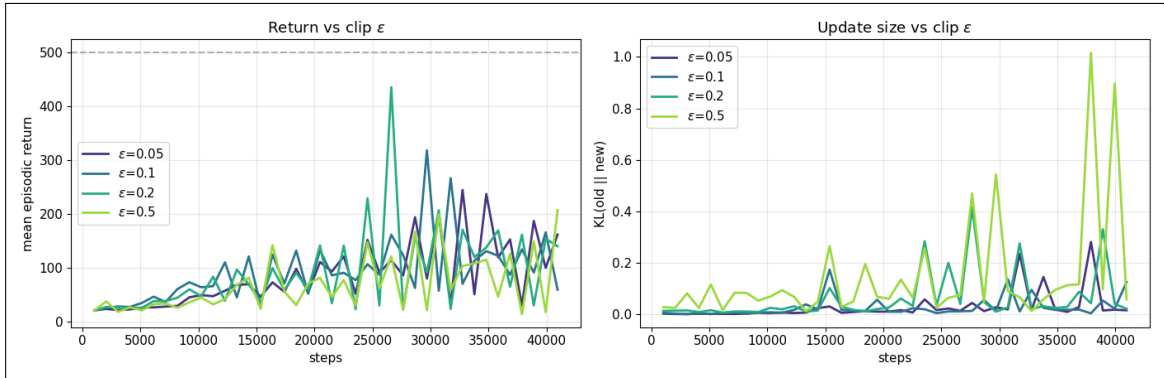


Figure 4: Effect of the clip range $\varepsilon \in \{0.05, 0.1, 0.2, 0.5\}$. Left: episodic return. Right: per-update KL. Larger ε allows larger KL steps per update, as expected.

5.2 GAE λ

The GAE parameter λ controls the bias-variance tradeoff in the advantage estimator. $\lambda = 0$ is the one-step TD residual (low variance, biased, heavily dependent on a good V_ϕ); $\lambda = 1$ is the Monte-Carlo advantage (unbiased, high variance); intermediate values such as 0.95 are the standard PPO choice. At this single-seed 40k-step budget, $\lambda = 0$ actually produced the highest final return in our run (Table 4), while the monotonic “intermediate is best” story failed to materialize. We attribute this to single-seed noise rather than a systematic effect: the differences between $\lambda = 0$ (188.8) and $\lambda = 0.95$ (139.6) are comparable to the iteration-to-iteration oscillation observed in the main comparison, and with only one seed we cannot attribute them to the value of λ itself.

Table 4: Effect of GAE λ (clipped PPO, seed 0, 40k steps).

λ	Final return
0.00	188.8
0.50	117.6
0.95	139.6
1.00	84.2

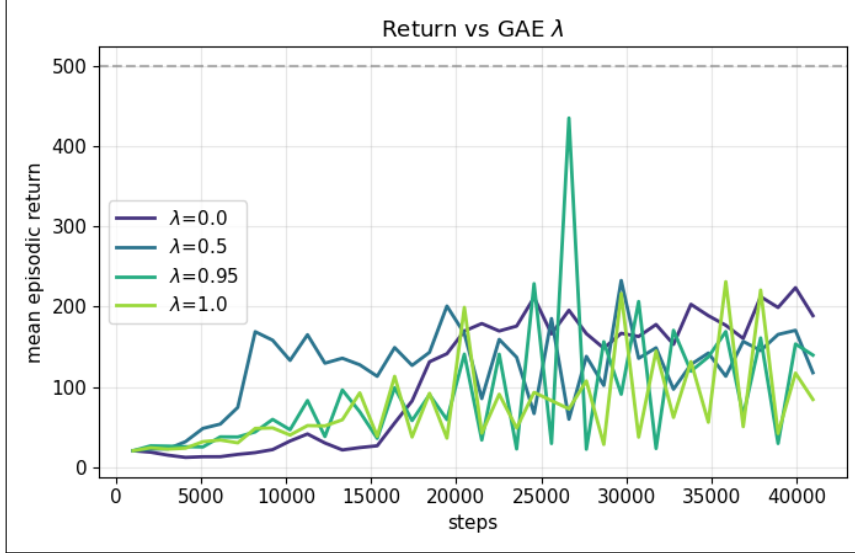


Figure 5: Effect of GAE $\lambda \in \{0.0, 0.5, 0.95, 1.0\}$ on the learning curve. The final-return ordering is not monotonic in λ at this single-seed budget.

5.3 Learning Rate

The learning-rate sweep produces a clean positive trend within the tested range (Table 5). Larger learning rates converge faster within the 40k-step budget, with $\text{lr} = 3 \times 10^{-3}$ hitting the 500-return cap. This does not contradict the usual PPO recommendation of $\text{lr} \approx 3 \times 10^{-4}$: on harder environments, large learning rates lose the stability race, but **CartPole-v1** is small enough that a fast aggressive optimizer finishes the task before any divergence issues accumulate.

Table 5: Effect of learning rate (clipped PPO, seed 0, 40k steps).

Learning rate	Final return
1×10^{-4}	88.6
3×10^{-4}	139.6
1×10^{-3}	330.0
3×10^{-3}	500.0

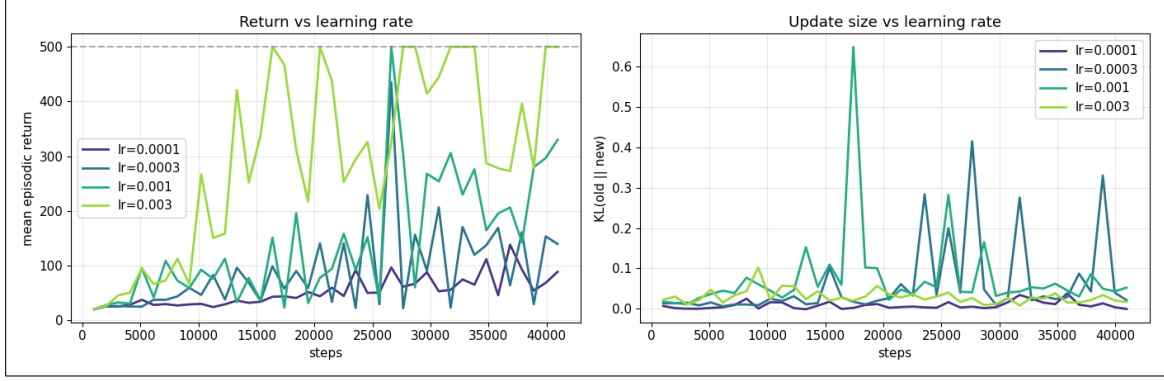


Figure 6: Effect of learning rate $lr \in \{1e-4, 3e-4, 1e-3, 3e-3\}$. Left: return. Right: per-update KL. Larger learning rates produce both higher returns and larger KL steps at this horizon.

5.4 PPO Epochs per Rollout

PPO epochs control how many gradient-descent passes are made over each rollout. One epoch is essentially vanilla policy gradient; many epochs amortize the cost of rollouts but push the policy further from θ_{old} on every pass, so the clip starts firing harder. The sweep (Table 6) shows a clean positive trend at this budget: more epochs means more learning progress per rollout, and $K = 20$ gives the highest final return among the values tested. The corresponding KL-per-update and clip-fraction panels (Figure 7) confirm that both grow with K as expected.

Table 6: Effect of PPO epochs per rollout (clipped PPO, seed 0, 40k steps).

K	Final return
1	35.2
4	58.8
10	139.6
20	247.0

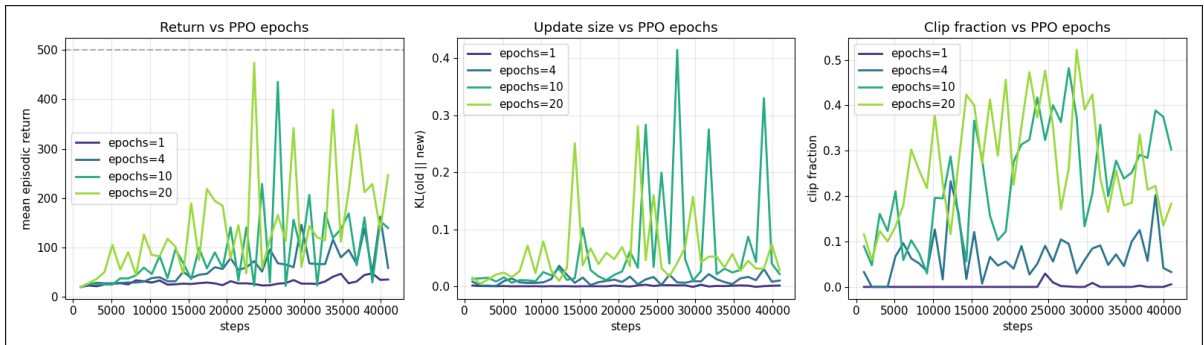


Figure 7: Effect of PPO epochs per rollout $K \in \{1, 4, 10, 20\}$. Left: return. Middle: per-update KL. Right: clip fraction. As K grows, both KL and clip fraction grow, and so do final returns at this training budget.

5.5 Entropy Coefficient

Adding $-c_e H[\pi]$ to the loss rewards the policy for being uncertain, encouraging exploration. On **CartPole-v1** the optimal policy is nearly deterministic, so a large entropy bonus actively hurts. The sweep (Table 7) confirms that the default $c_e = 0$ achieves the highest final return

among the tested values, and that $c_e \in \{0.05, 0.1\}$ clearly degrades performance. The relatively small gap between $c_e = 0$ (139.6) and $c_e = 0.01$ (110.7) is within single-seed noise.

Table 7: Effect of entropy coefficient (clipped PPO, seed 0, 40k steps).

c_e	Final return
0.00	139.6
0.01	110.7
0.05	33.6
0.10	78.5

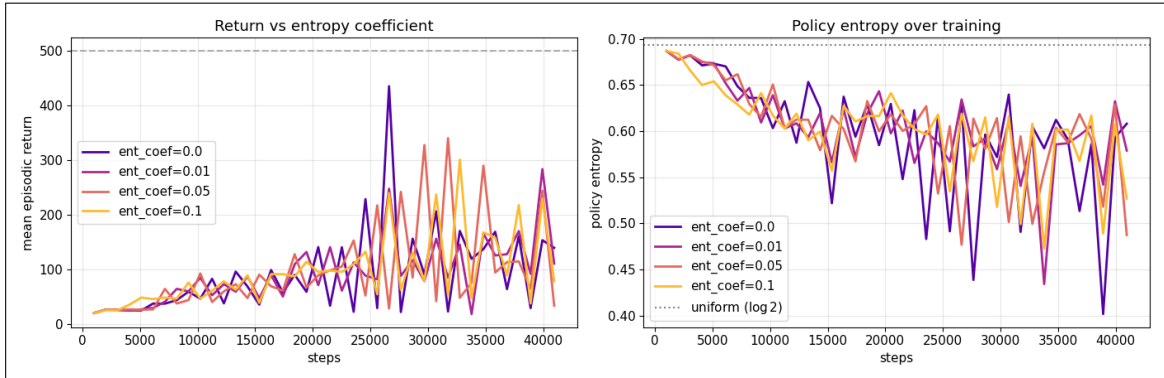


Figure 8: Effect of entropy coefficient $c_e \in \{0.0, 0.01, 0.05, 0.1\}$. Left: return. Right: policy entropy over training. Large c_e prevents the policy from becoming deterministic, which caps the achievable return.

5.6 Summary of Ablations

The sweeps reveal two qualitatively different patterns. Learning rate and PPO epochs produce monotonic trends within the ranges tested: higher is better at a 40k-step budget on this task. Clip range ε , GAE λ , and the entropy coefficient c_e do not produce clean monotonic trends; the differences between adjacent settings are often smaller than the observed iteration-to-iteration noise, and with a single seed we cannot cleanly separate signal from noise. The standard PPO defaults ($\varepsilon = 0.2$, $\lambda = 0.95$, $c_e = 0$) sit in the middle of the ranges tested and, while not optimal at this specific budget and seed, are widely reported as robust choices across harder tasks. Our sweeps reinforce one important practical lesson: single-seed sweeps in reinforcement learning are unreliable for making fine-grained decisions, and any reported difference should be judged against the seed-to-seed variance of the main comparison.

6 Episode-Length Evolution

On `CartPole-v1` the reward is +1 per timestep, so episodic return equals episode length. Tracking the evolution of per-rollout episodic returns is therefore a direct readout of how long the agent survives over training.

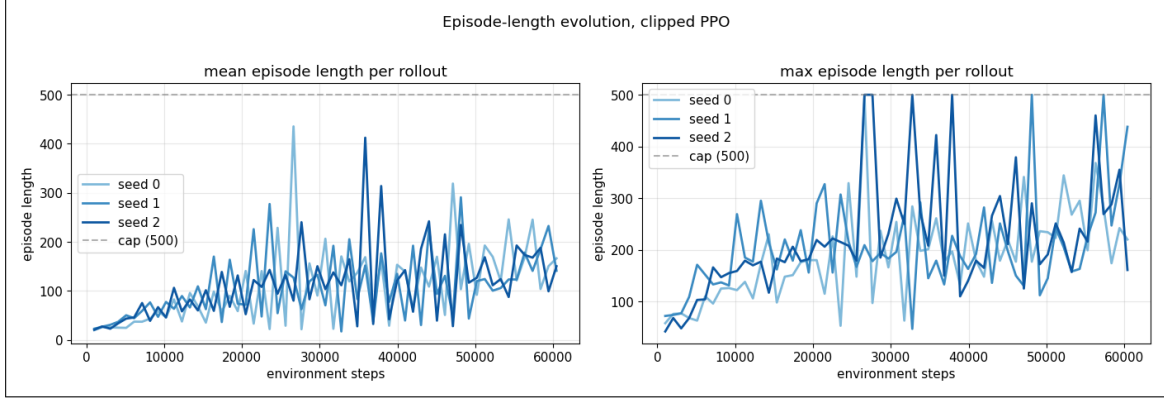


Figure 9: Per-rollout episode length for each clipped-PPO seed. Left: mean episode length per rollout. Right: maximum episode length per rollout. Both curves are noisy: the clipped agent repeatedly climbs above 200 and then drops back, rather than monotonically approaching the 500-step cap at this training budget.

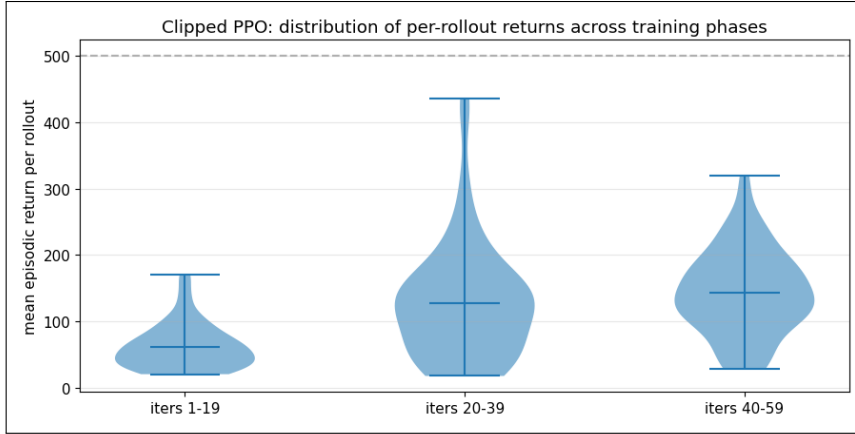


Figure 10: Distribution of per-rollout mean episodic returns across three training phases (early, middle, late thirds of training), aggregated over the three clipped-PPO seeds. The distribution shifts upward across phases but retains a long lower tail, reflecting the noisy climb visible in the per-seed curves.

7 Learned Value Function

CartPole-v1 has a four-dimensional state. We visualize the critic’s output as a 2D slice, holding cart position and cart velocity at zero and sweeping pole angle and pole angular velocity. A well-trained critic is expected to place a high $V_\phi(s)$ near the origin (pole upright and still) and lower values as the pole tilts or spins fast, since those states are closer to termination.

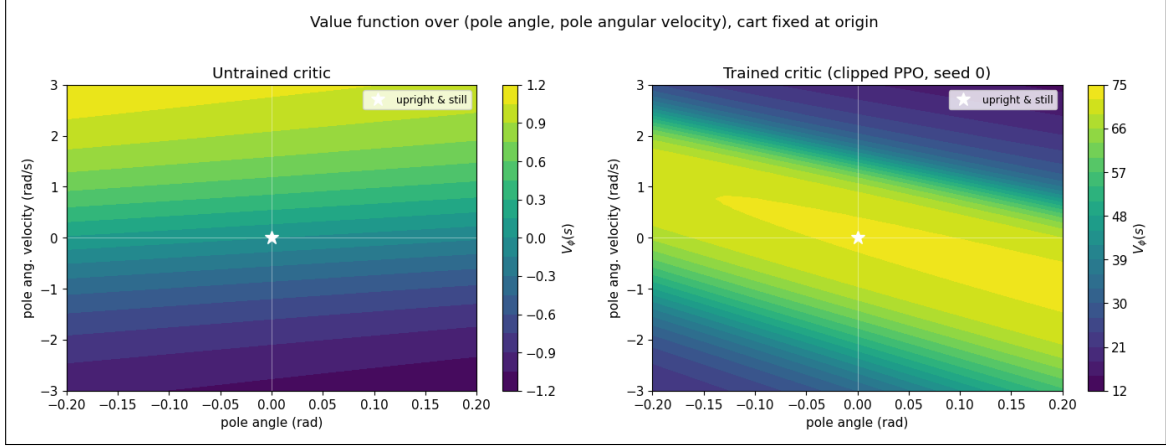


Figure 11: Learned value function $V_\phi(s)$ as a function of pole angle (x-axis) and pole angular velocity (y-axis), with cart position and cart velocity held at zero. Left: an untrained critic (random initialization) produces an essentially flat landscape. Right: the trained critic (clipped PPO, best seed) peaks near the origin (the upright-and-still state, marked with a white star) and decays as the pole tilts or spins.

The trained heatmap is a clean qualitative sanity check on the critic. The near-flat output of the untrained network is the expected baseline: it has seen no data yet, and its output reflects only orthogonal initialization.

8 Rendering and Qualitative Evaluation

Gymnasium can render the environment as an RGB image at each step (`render_mode="rgb_array"`). We pick the best clipped-PPO seed, roll out one greedy episode, and save the resulting frames as an animated GIF (`clipped_agent.gif`). We do the same for the best unclipped seed (`unclipped_agent.gif`).

Greedy evaluation returns from the saved models are reported in Table 8. On this particular evaluation seed, the clipped agent held the pole for 176 steps; the unclipped agent held it for the full 500-step cap. This is consistent with the main comparison: the unclipped variant can reach peak performance on favourable seeds, while the clipped variant is more stable but lower-peaking at this training budget.

Table 8: Greedy evaluation returns (single rollout, evaluation seed 123) of the best across-seeds checkpoint of each variant.

Variant	Best seed	Greedy return
Clipped PPO	0	176
Unclipped PPO	0	500

A single clip of a trained agent does not convey *learning*. To illustrate the progression, we train one fresh agent with checkpointing at $\{0, 8,000, 20,000, 60,000\}$ environment steps, roll out a greedy episode from each checkpoint with the same evaluation seed, and stitch the resulting clips into a single annotated GIF (`progression.gif`). The per-checkpoint greedy returns (evaluation seed 42) are:

Table 9: Greedy-evaluation survival time (evaluation seed 42) at four training checkpoints of one clipped-PPO training run (seed 0).

Training steps	Survival steps
0	103
8,000	198
20,000	19
60,000	182

Interestingly, the progression is not monotonic: the randomly initialized policy happens to survive 103 steps on this evaluation seed (the orthogonal-initialized policy is effectively near-uniform, which is non-catastrophic on `CartPole`); training to 8k steps improves survival to 198; training to 20k steps actually produces a temporary regression (19 steps), which is consistent with the seed-0 training log showing collapses around this region; and training to the full 60k steps recovers to 182 steps. This illustrates the noisy, non-monotonic nature of single-seed on-policy learning: the training-time mean return averages over many episodes per rollout, but a single greedy evaluation against a fixed evaluation seed picks out a particular slice of the policy that may be locally worse even when the overall policy is improving.

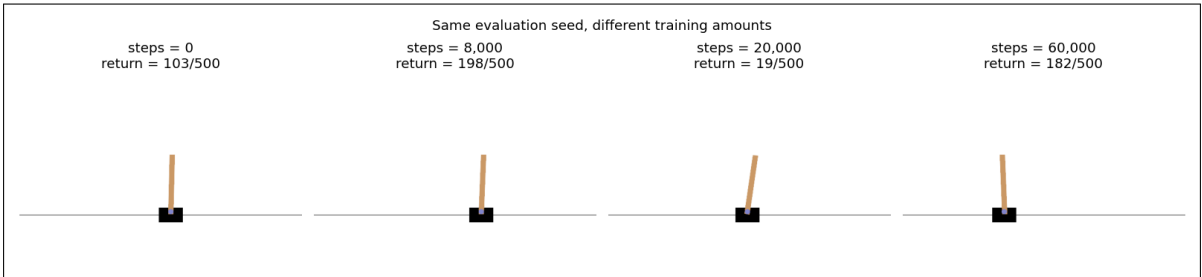


Figure 12: One frame from each training checkpoint, evaluated under the same seed. Survival times are non-monotonic ($103 \rightarrow 198 \rightarrow 19 \rightarrow 182$ steps), which is typical of single-seed on-policy RL.

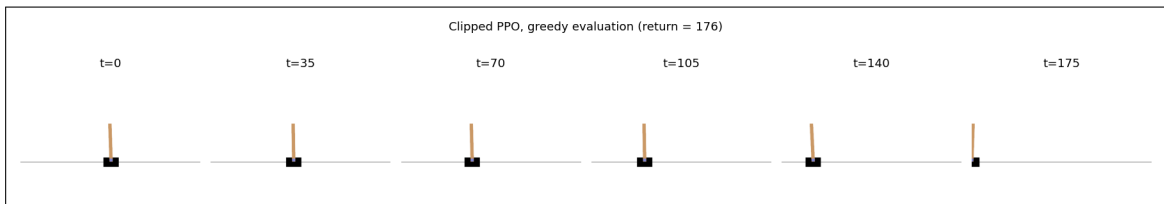


Figure 13: Six evenly-spaced frames from the final clipped-PPO greedy evaluation (return 176).

9 Discussion

What the main comparison actually shows. The clipped and unclipped objectives behave very differently at a matched 60k-step budget. The unclipped objective hits a higher mean return but with an order-of-magnitude larger across-seed standard deviation. The clipped objective produces a tight, predictable band at the cost of not capitalizing on lucky updates. The diagnostics pin this down to per-update KL: the clipped agent stays in a narrow KL band, the unclipped agent allows KL spikes that correspond exactly to the collapse events visible in its learning curve. In a production setting where reliability matters more than a single lucky run, the clipped variant is clearly preferable.

Single-seed sweeps are noisy. Three of the five sweep results (ε , λ , c_e) do not produce a clean monotonic ranking in the narrow range tested. This is not a failure of the theory (the standard PPO folklore is well supported on harder tasks over more seeds); it is a property of the evaluation protocol. A 40k-step single-seed run on **CartPole-v1** is too short and too noisy to distinguish small differences between hyperparameter settings, and the honest conclusion is that only the learning-rate and PPO-epochs sweeps produced a signal clearly above the noise at this budget.

Why the unclipped variant can beat the clipped one on CartPole-v1. The task is small and episodes terminate quickly, so the unclipped surrogate’s occasional large updates can either collapse the policy or push it past the “just balance the pole” threshold in a single rollout. With only three seeds, the mean across seeds mixes both outcomes. On harder tasks with longer episodes and richer reward structure, the collapse mode is much more common than the jackpot mode, and the clipped objective’s stability advantage translates into a clean performance advantage as well.

Limitations. The main comparison uses only three seeds; the sweeps use a single seed. Both numbers are honest but noisy, and the qualitative ordering statements should be trusted far more than the specific final-return values. A more rigorous study would run each setting over 5 to 10 seeds, report per-seed final returns rather than means, and compute across-seed confidence intervals. The training budget (60k steps) is also short enough that not every seed has reached the 500-return cap.

10 Conclusion

We implemented PPO from scratch with a clear separation of policy network, value network, rollout loop, advantage estimator, and loss function; compared the clipped and unclipped surrogate objectives on **CartPole-v1**; ran ablations over the principal hyperparameters; visualized the learned value function; and rendered the trained policy. The key findings are:

1. Under matched hyperparameters across three seeds, the clipped variant is markedly more stable than the unclipped variant (across-seed std 10.4 vs. 175.4 at the final iteration). The unclipped variant actually reaches a higher mean return (376.0 vs. 152.8) because some seeds hit the 500-return cap; but other seeds collapse, which is exactly the failure mode that the clip is designed to prevent.
2. The diagnostics trace the difference to the per-update KL divergence: the clip keeps KL in a narrow band, the unclipped surrogate allows spikes that correspond to the collapse events in the learning curve.
3. Learning-rate and PPO-epoch sweeps produce clean monotonic trends at this single-seed 40k-step budget; ε , λ , and c_e sweeps do not. The standard defaults ($\varepsilon = 0.2$, $\lambda = 0.95$, $c_e = 0$) are not optimal at this specific budget but are reported to be robust on harder tasks.
4. The critic learns a sensible value landscape, with $V_\phi(s)$ peaking near the upright-and-still state and decaying as the pole tilts or spins.
5. Rendered rollouts at four training checkpoints show that single-seed greedy evaluation can be non-monotonic in training steps ($103 \rightarrow 198 \rightarrow 19 \rightarrow 182$), which is a useful reminder that per-rollout training returns and single-seed greedy evaluations are noisy proxies for policy quality.

References

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [2] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-Dimensional Continuous Control Using Generalized Advantage Estimation,” *Proc. ICLR*, 2016.
- [3] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, “Trust Region Policy Optimization,” *Proc. ICML*, 2015.
- [4] M. Towers et al., “Gymnasium,” <https://gymnasium.farama.org>, 2023.
- [5] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Proc. NeurIPS*, 2019.